

Database Management Systems (DBMS)

Definition Of Database

- A database (DB) is a system used to store, manage, and organize data.
- It helps you save data permanently so it can be retrieved, updated, and deleted easily.

Database Management System

Most popularly used DBMS are as follows:

- MySQL
- Microsoft SQL Server
- Oracle DBMS
- PostgreSQL
- MongoDB
- SQLite

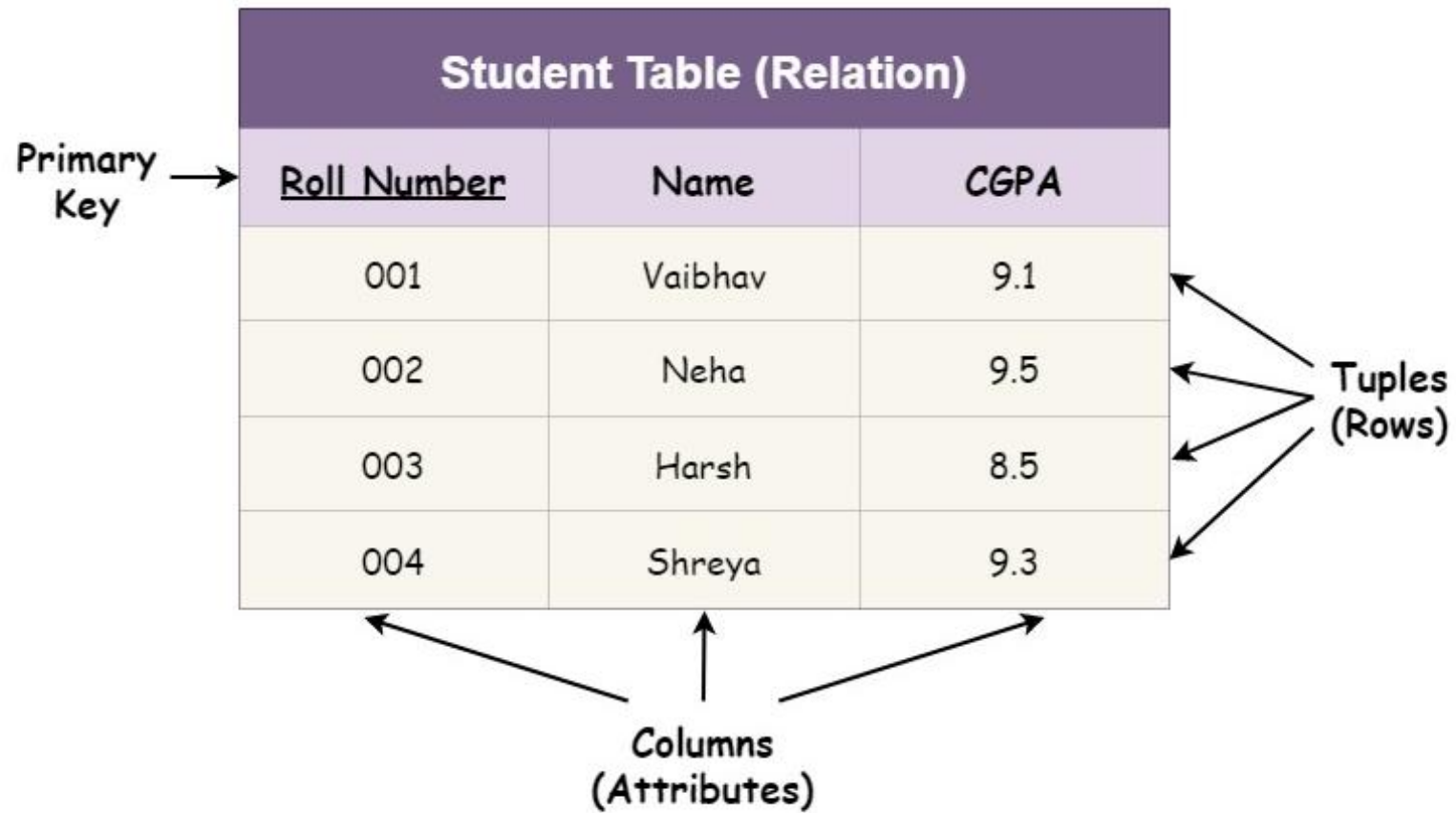
Benefits of the database approach

1. Minimal Data Redundancy
2. Improved Consistency of Data
3. Improved Data Sharing
4. Improved Data Integrity
5. Security can be enforced
6. Transaction support can be provided
7. Increased Productivity of Application Development
8. Enforcement of Standards

Relational Model

- The relational Model was proposed by E.F. Codd
- A relational database stores data in the form of relations (tables).
- Information is stored in the form of rows and columns. The basic structure of this model is tables. Generally, tables are called relations in the relational model.
- Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE, and AGE
- After designing the conceptual model of the Database using ER diagram, we need to convert the conceptual model into a relational model which can be implemented using any RDBMS language like Oracle SQL, MySQL, etc.

Relational Model



Relational Data Structure

- **Relation:** A relation is a *table* with columns and rows.
 - An RDBMS requires only that the database be perceived by user as tables at the external and conceptual levels.
 - It does not apply to the physical structure of database which can be implemented using a variety of storage structures such as sequential files, pointer chains etc.
- **Attribute:** An attribute is a *named column* of a relation.
 - For example, the attributes in Customer relation are custid, custname, address, telno, creditlimit etc.
- **Domain:** A domain is the *set of allowable values* for one or more attributes.
 - Domains may be distinct for each attribute or two or more attributes may be defined on the same domain.
 - e.g. domain for *custid* in customer table is 0 to 9999 where as domain for *creditlimit* is 0 to 100000.

Relational Data Structure

- Similarly, the domain of attribute *loan-number* might be the set of all strings of the form *L-n* where *n* is a positive integer.
- The domain of *City* in *Branch* table can be defined as the set of all character strings of maximum size 15.
- **Tuple:** A tuple is a *row* of a relation
 - In the Branch relation, each row contains four values, one for each attribute.
- **Intension of a relation:** The structure of a relation, together with a specification of the domains and any other restrictions on possible values.
 - It is usually fixed unless the meaning of a relation is changed to include additional attributes.

Relational Data Structure

- **Extension of a relation:** The information about the tuples are called the extension (or state) of a relation which changes over time.
- **Degree of a relation:** number of attributes in a relation.
 - Unary Relation (with only one attribute)
 - Binary Relation (with two attributes)
 - Ternary Relation (with three attributes)
 - n-ary Relation (with n attributes)
 - ❖ Degree is a property of the intension of a relation.
 - ❖ *What is the degree of branch and staff relations in fig 4?*
- **Cardinality:** Number of tuples in a relation.
 - The cardinality changes as tuples are added or deleted.
 - Cardinality is a property of the extension of a relation.
 - *What is the cardinality of branch and staff relations in fig 4?*
- **Relational database:** A collection of *normalized relations* with distinct relation names.

Terminology Comparison

Logical Term	Physical Relational Term	File System Term
Relation	Table	File
Tuple	Row	Record
Attribute	Column	Field

Important Terminologies (1)

- **Entity** – An entity represents a particular type of object in the real world.
- **Entity set** – Sets of entities of the same type which share the same properties are called entity Sets.
- **Attribute** – An attribute is a characteristic of an entity. For example: Object is STUDENT and its attribute are Roll no, Branch, Setmarks() in the Student class
- **Constraints** – A constraint is a restriction placed on the data. It is helpful to ensure data integrity.
- **Relationship** – A relationship describes an association among entities.

Important Terminologies (2)

- **Tuple:** Each row in the relation is known as a tuple.
- **Relation Schema:** A relation schema represents the name of the relation with its attributes. e.g.; STUDENT (ROLL_NO, NAME, ADDRESS, PHONE, and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation, it is called Relational Schema.
- **SQL:** (Structure Query Language) is a domain-specific language used in programming and designed for managing data held in a relational database management system (RDBMS), or for stream processing in a relational data

Database Architecture

- A **Database Architecture** is a representation of DBMS design.
- A DBMS architecture allows dividing the database system into individual components that can be independently modified, changed, replaced, and altered.
- It also helps to understand the components of a database.
- A Database stores critical information and helps access data quickly and securely. Therefore, selecting the correct Architecture of DBMS helps in easy and efficient data management.

Types of DBMS Architecture

There are mainly three types of DBMS architecture:

- One Tier Architecture (Single Tier Architecture)
- Two Tier Architecture
- Three Tier Architecture

1-Tier Architecture

- **1 Tier Architecture** in DBMS is the simplest architecture of Database in which the client, server, and Database all reside on the same machine.
- This type of architecture is rarely used in production.

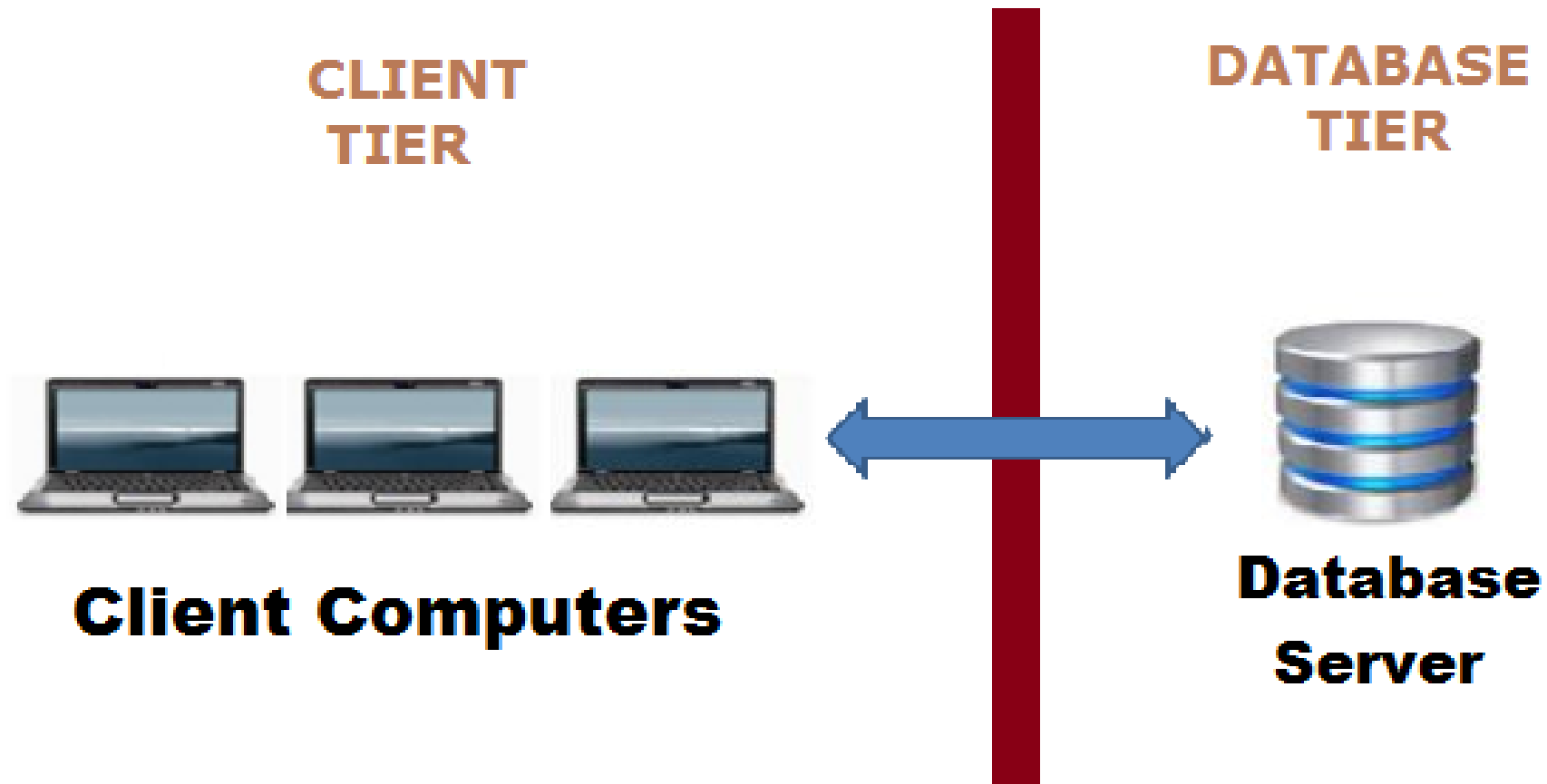


Single Tier Architecture

2-Tier Architecture

- A **2 Tier Architecture** in DBMS is a Database architecture where the presentation layer runs on a client (PC, Mobile, Tablet, etc.), and data is stored on a server called the second tier
- Two tier architecture provides added security to the DBMS as it is not exposed to the end-user directly.
- It also provides direct and faster communication.

2-Tier Architecture



3-Tier Architecture

- A **3 Tier Architecture** in DBMS is the most popular client server architecture in DBMS
- Three Tier architecture contains a presentation layer, an application layer, and a database server.
- 3-Tier database Architecture design is an extension of the 2-tier client-server architecture. A 3-tier architecture has the following layers:
 - Presentation layer (your PC, Tablet, Mobile, etc.)
 - Application layer (server)
 - Database Server

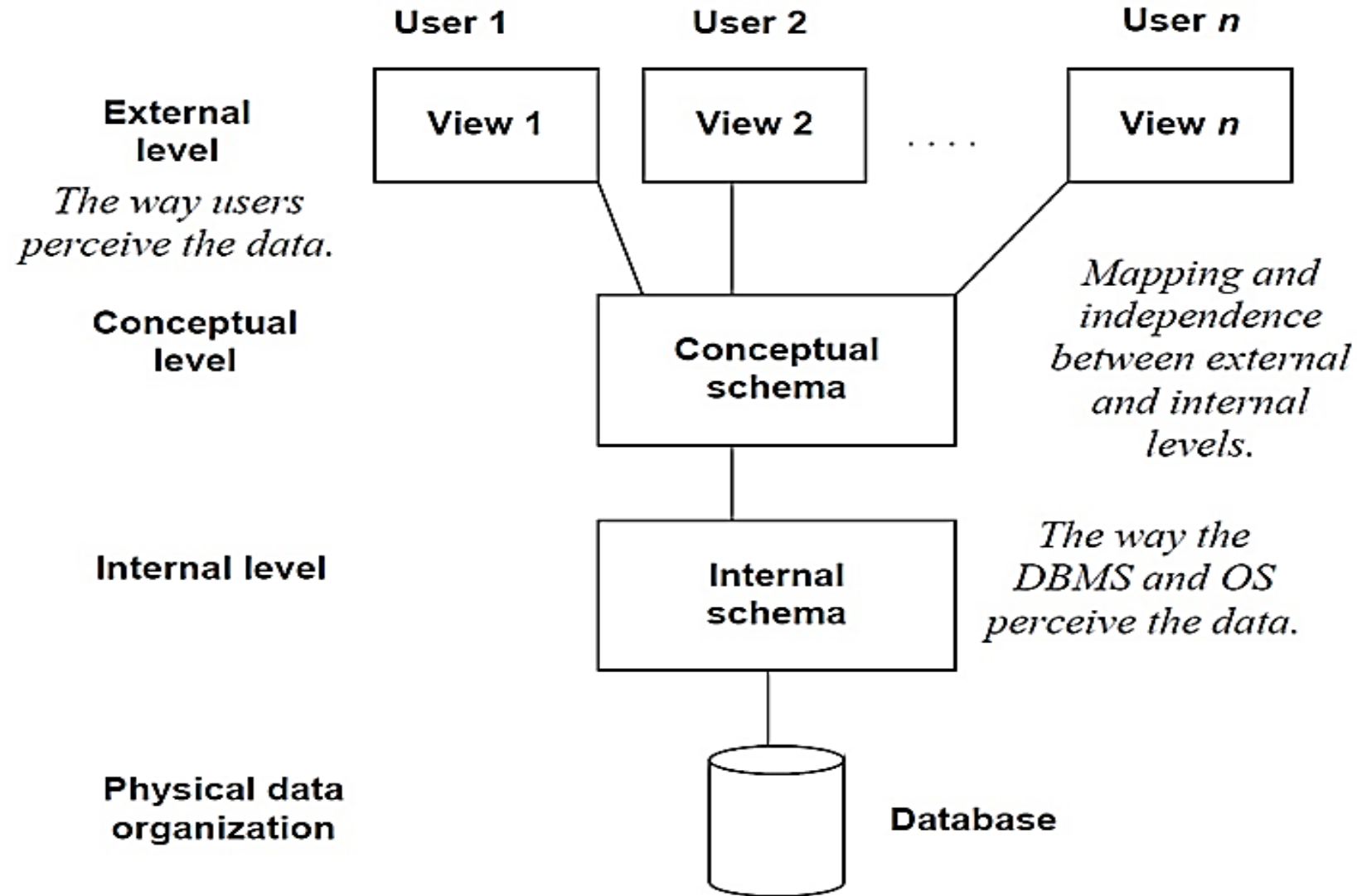
DBMS 3 tier Architecture



Three schema Architecture

- The three schema architecture is also called ANSI/SPARC architecture or three-level architecture.
- This framework is used to describe the structure of a specific database system.
- The three schema architecture is also used to separate the user applications and physical database.
- The three schema architecture contains three-levels. It breaks the database down into three different categories.
- Internal schema
- Conceptual schema
- External schema

Three schema Architecture



Internal schema/physical schema

- The internal level has an internal schema which describes the physical storage structure of the database.
- It uses the physical data model. It is used to define that how the data will be stored in a block.
- The internal level is generally is concerned with the following activities:
 1. Storage space allocations.
 2. Access paths.
 3. Data compression and encryption techniques

STORED_EMPLOYEE record length 60

Empno : 4 decimal offset 0 unique

Ename : String length 15 offset 4

Salary : 8,2 decimal offset 19

Deptno : 4 decimal offset 27

Post : string length 15 offset 31

Internal view

Conceptual Level

- The conceptual schema describes the design of a database at the conceptual level. Conceptual level is also known as logical level.
- The conceptual level describes what data are to be stored in the database and also describes what relationship exists among those data.
- In the conceptual level, internal details such as an implementation of the data structure are hidden.
- Programmers and database administrators work at this level.

Global view

EMPLOYEE

Empno : Integer(4) Key

Ename : String(15)

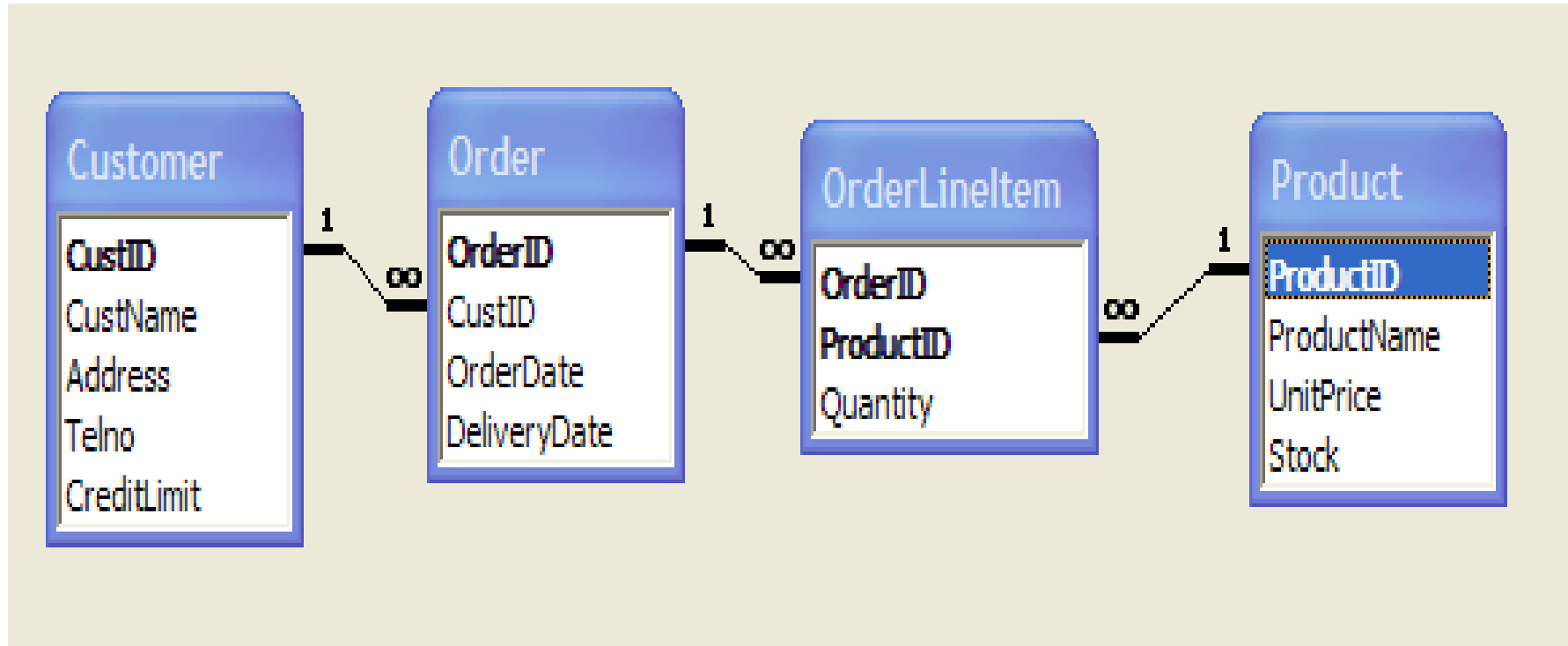
Salary : String (8)

Deptno : Integer(4)

Post : String (15)

Conceptual Schema Using ER Model

- Conceptual schema can be depicted in graphical format using *ER or Object Modeling notations*;
- This type of conceptual schema is usually called a data model.



Conceptual Schema Using DDL

<pre>CREATE TABLE CUSTOMER (CUSTID NUMBER(4) PRIMARY KEY, CUSTNAME VARCHAR2(25) NOT NULL, ADDRESS VARCHAR2(50), TELNO VARCHAR2(20), CREDITLIMIT NUMBER(6) CHECK (CREDITLIMIT BETWEEN 0 AND 100000));</pre>	<pre>CREATE TABLE ORDER (ORDERID NUMBER(4) PRIMARY KEY, CUSTID NUMBER(4), ORDERDATE DATE, DELIVERYDATE DATE, FOREIGN KEY (CUSTID) REFERENCES CUSTOMER (CUSTID));</pre>
<pre>CREATE TABLE ORDERLINEITEM (ORDERID NUMBER(4), PRODUCTID NUMBER(4), QUANTITY NUMBER(5), PRIMARY KEY (ORDERID, PRODUCTID), FOREIGN KEY (ORDERID) REFERENCES ORDER(ORDERID), FOREIGN KEY (PRODUCTID) REFERENCES PRODUCT(PRODUCTID));</pre>	<pre>CREATE TABLE PRODUCT (PRODUCTID NUMBER(4) PRIMARY KEY, PRODUCTNAME VARCHAR2(25) NOT NULL, UNITPRICE NUMBER(4), STOCK NUMBER(4) CHECK (STOCK BETWEEN 0 AND 1000));</pre>

External Level/view schema

- Each view schema describes the database part that a particular user group is interested and hides the remaining database from that user group.
- The view schema describes the end user interaction with database systems.
- At the external level, a database contains several schemas that sometimes called as subschema. The subschema is used to describe the different view of the database.

External
View

Empno	Ename
-------	-------

Empno	Ename	Salary	DeptNo
-------	-------	--------	--------

Properties of Relations

1. Every relation has a name that is *distinct* from all other relation names in the relational schema.
2. Each cell of the relation contains exactly *one atomic (single) value*.
 - It is illegal to store two telephone numbers of customer in a single cell.
 - In other words, relations do not contain repeating groups.
 - A relation that satisfies this property is called **normalized**.
3. Each attribute in a relation should have a *distinct name*.
4. The values of an attribute are all from the *same domain*.
5. Each tuple is distinct; there are *no duplicate tuples*.
 - Enforced using primary key

Properties of Relations (cont'd)

6. The *order of attributes* has no significance
7. The *order of tuples* has no significance; theoretically. (In practice, order may affect the efficiency of accessing tuples)

Relational Keys

- Since a relation cannot have duplicate tuples,
 - we need to be able to identify one or more attributes
 - that uniquely identify each tuple in a relation.
- ❖ Super Key
 - ❖ Candidate Key
 - ❖ Primary Key
 - ❖ Alternate Key
 - ❖ Foreign Key

Relational Keys

1. Super Key (SK)

- *An attribute or set of attributes that **uniquely identifies** a tuple within a relation.*
- A superkey may contain additional attributes that are not necessary for unique identification.
- e.g.
 - i. OrderID and Orderdate in Order relation is a super key.
 - ii. OrderID, ProductID, Quantity in OrderLineItem relation is a super key.

2. Candidate Key (CK)

- *A super key such that **no proper subset** is a super key within the relation.*
- e.g.
 - i. OrderID in Order relation is a candidate key.
 - ii. OrderID, ProductID in OrderLineItem relation is a candidate key.

Relational Keys

2. Candidate Key (cont'd)

- A candidate key, K , for a relation R , has two properties:-
 - ❖ Uniqueness – in each tuple of R , the values of K uniquely identify that tuple;
 - ❖ Irreducibility – no proper subset of K has the uniqueness property
- There may be *several candidate keys* for a relation.
- Consider the *Product* table. Besides the *ProductID*, the *ProductName* may also be a candidate key.
- When a key consists of more than one attribute, we call it a *composite key*.
 - ❖ In the *OrderLineItem* relation, *orderid* and *productid* together form the composite candidate key.

Relational Keys

3. Primary Key (PK)

- *The candidate key that is selected to identify tuples uniquely within the relation.*
- In the worst case, the entire set of attributes could serve as the primary key but usually some smaller subset is sufficient to distinguish the tuples.
- The primary key should be chosen such that its attributes are never, or very rarely changed.
- For instance, the address field of a person should not be part of the primary key, since it is likely to change.
- Social-security numbers (or CNIC numbers), on the other hand, are guaranteed to never change.

Relational Keys

4. Alternate Key

- The CKs that are *not selected to be the PK*.
- For Product relation, if we choose *productid* as the PK, *productname* would then be the alternate key.

5. Foreign Key

- An attribute or set of attributes, within one relation that *matches the PK* of some (possibly the same) relation.
- For example, the inclusion of *custid* in both Customer and Order relations links each Customer to the details of Order given by that customer.
- In the Order relation, *custid* is the foreign key.

Customer(CustID, ...) **Order(OrderID, CustID, ...)**

STUDENT

ID

Name

Address

Course

PERSON

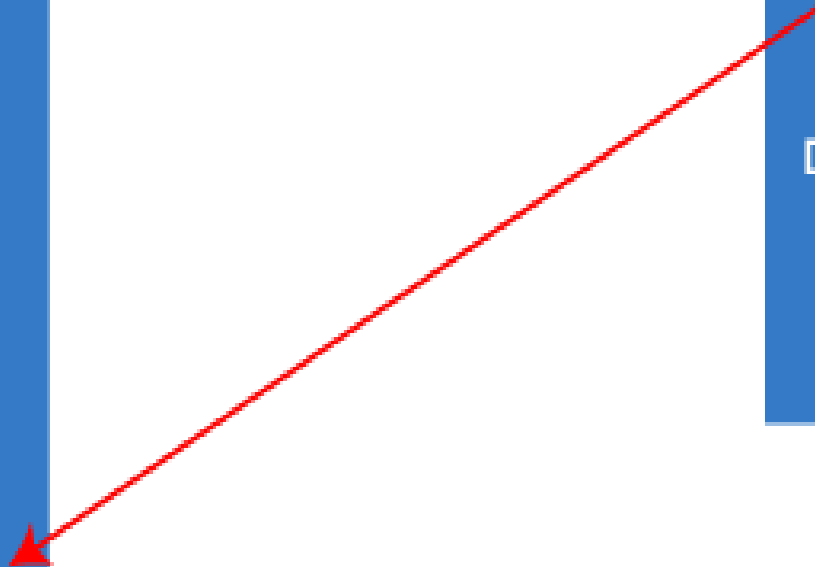
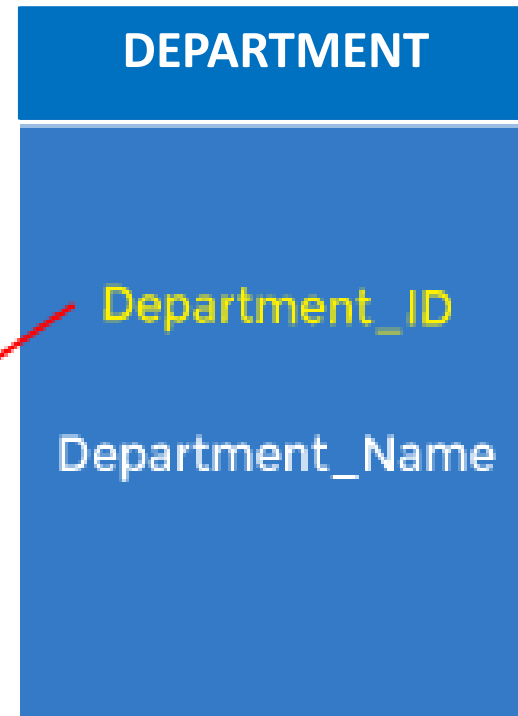
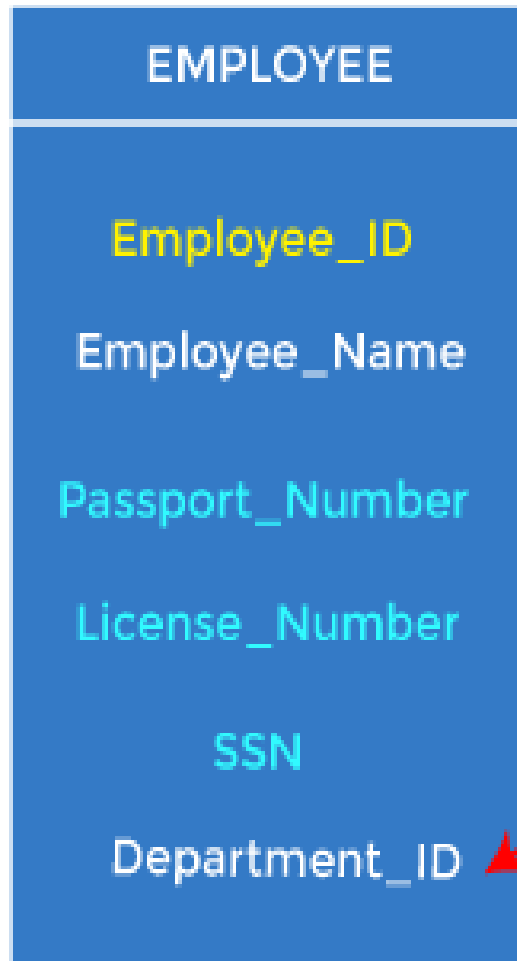
Name

DOB

Passport, Number

License_Number

SSN



Foreign Key (Example 2: Teachers and Subjects)

Teachers Table:

TeacherID	Name
T001	Miss Ali
T002	Sir Asim

Subjects Table:

SubjectID	TeacherID	SubjectName
S001	T001	English
S002	T002	Math

Here, TeacherID in the Subjects Table is the foreign key

A foreign key is like a pointer that says:

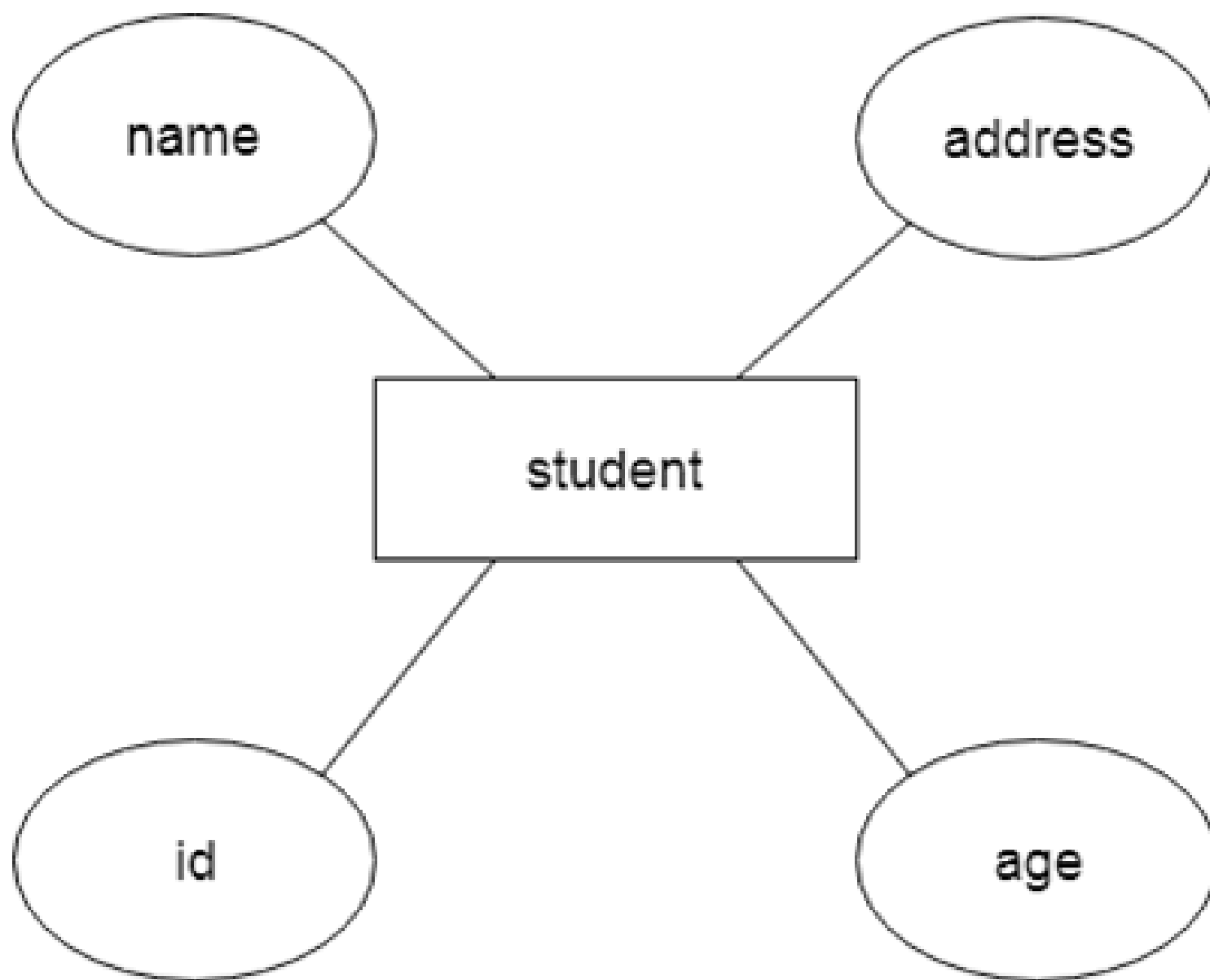
“This record belongs to that person or thing in another table.”

ER Model

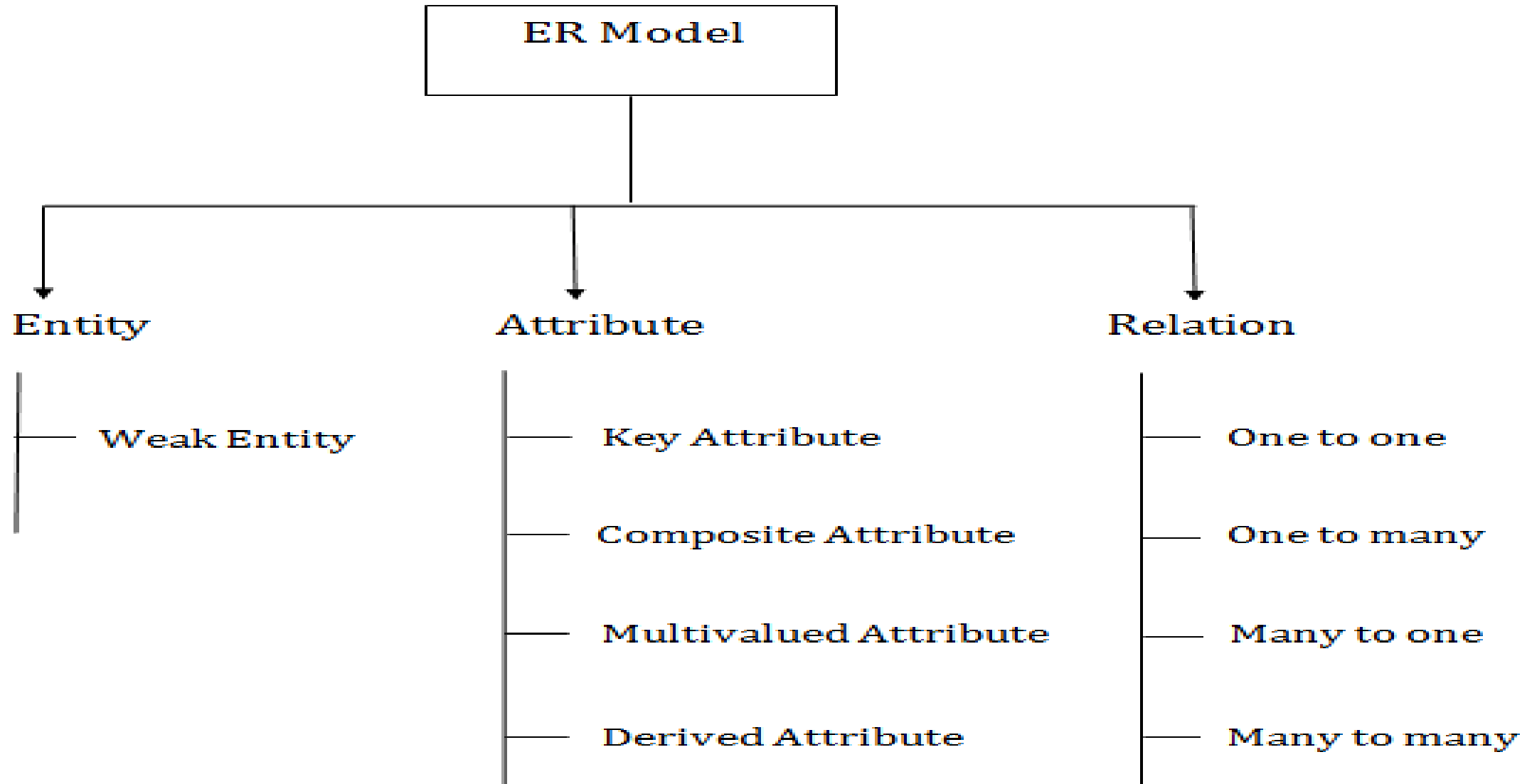
- ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database. It also develops a very simple and easy to design view of data.
- In ER modeling, the database structure is portrayed as a diagram called an entity-relationship diagram.

Example:

Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc and there will be a relationship between them.

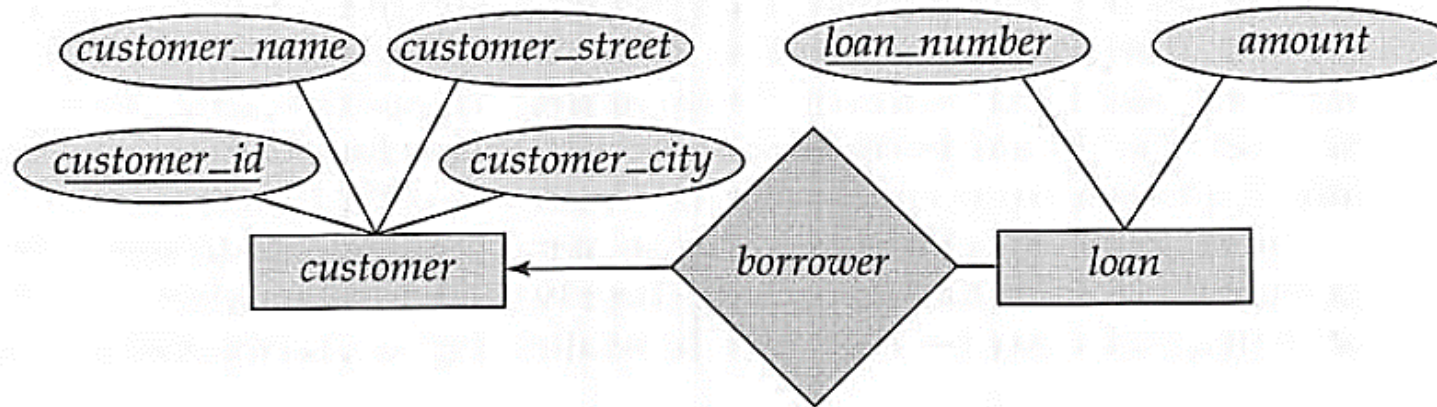


Components Of ER Diagram



Entity-Relationship Model

- The Entity-Relationship model is a *detailed, conceptual representation* of the data for an organization.
- It models the
 - real world *entities* in the business environment,
 - the *relationships* (or associations) among those entities and
 - their *attributes* (or properties)
 - using E-R diagrams, which is a graphical representation of an E-R model.



- The E-R model was introduced by **Chen** (1976).

COMPONENT OF ER model

Attribute :

- The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.

Example: id, age, contact number, name, etc. can be attributes of a student.

Key Attribute :

The key attribute is used to represent the main characteristics of an entity. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.

COMPONENT OF ER model

Composite Attribute:

- An attribute that composed of many other attributes is known as a composite attribute. The composite attribute is represented by an ellipse, and those ellipses are connected with an ellipse.

For example, student's address will contain, house no., street name, zipcode etc

Multivalued Attribute :

- An attribute can have more than one value. These attributes are known as a multivalued attribute. The double oval is used to represent multivalued attribute.

For example, a student can have more than one phone number

COMPONENT OF ER model

- **Derived Attribute** : An attribute that can be derived from other attribute is known as a derived attribute. It can be represented by a dashed ellipse.

For example, A person's age changes over time and can be derived from another attribute like Date of birth.

- **Relationship:**

A relationship is used to describe the relation between entities. Diamond or rhombus is used to represent the relationship.

E-R Diagrams

- Fig 4 shows how composite, multivalued and derived attributes can be represented in the E-R notation.
- Here, a composite attribute *name* with component attributes *first_name*, *middle_initial*, and *last_name* replaces the simple attribute *customer_name* of customer.
- Also, a composite attribute *address* whose component attributes are *street*, *city*, *state* and *zip_code* replaces the attributes *customer_street* and *customer-city* of customer.
- Fig 4 also illustrates phone-number, depicted by a double ellipse, and *age*, depicted by a dashed ellipse.

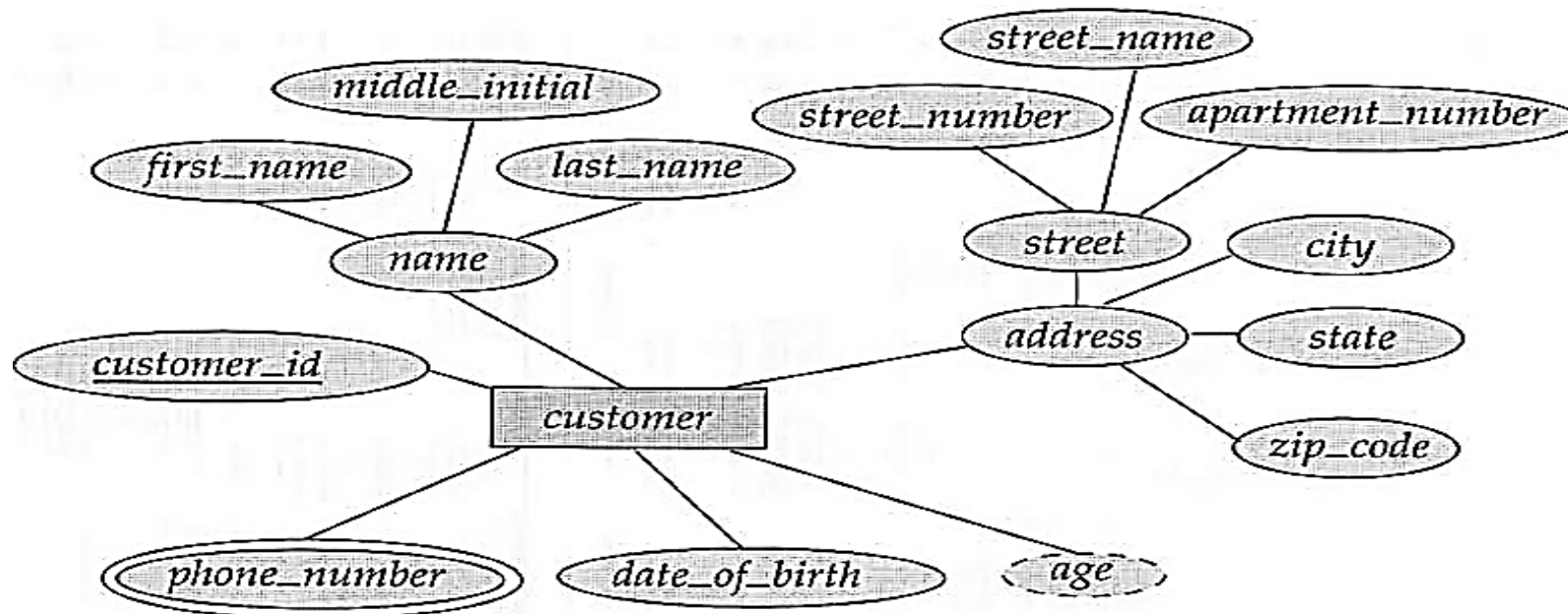


Fig 4

COMPONENTS OF ER MODEL

Types of relationship are as follows:

➤ One-to-One Relationship:

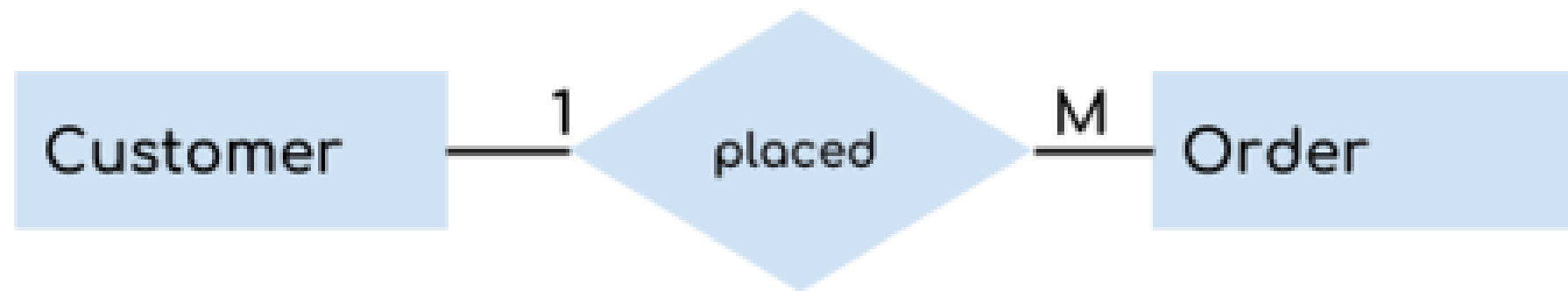
When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport and a passport is given to one person.



COMPONENTS OF ER MODEL

➤ One-to-many relationship:

When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship. For example – a customer can place many orders but a order cannot be placed by many customers.



COMPONENTS OF ER MODEL

➤ Many-to-one relationship:

When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship. For example – many students can study in a single college but a student cannot study in many colleges at the same time.



COMPONENTS OF ER MODEL

➤ Many-to-many relationship:

When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship. For example, a student can be assigned to many projects and a project can be assigned to many students.



Degree of relationship set

- The number of entity types which took part in the entity relationship is called the degree of relationships.

There are three different types of degree of relationships, they are as follows –

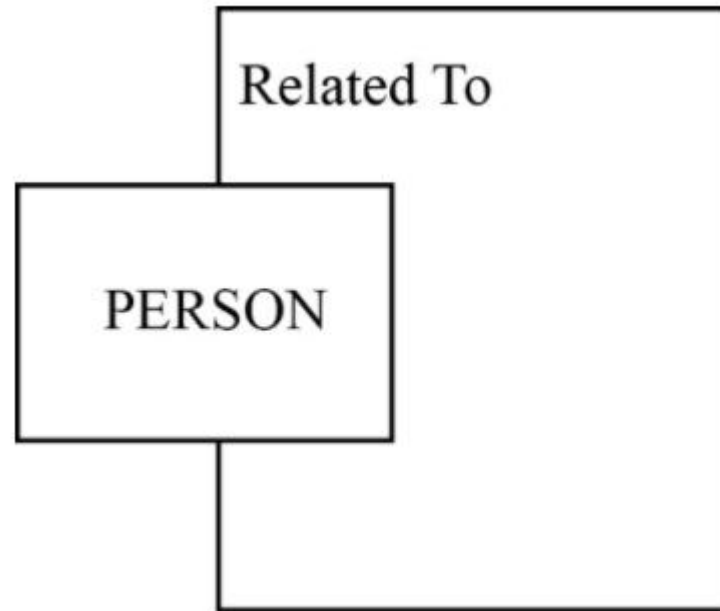
- Unary relationship
- Binary relationship
- Ternary relationship

Unary relationship

- It is the relationship between the instances of a single entity type. It is also called a recursive relationship.

Example

- A person is married is a one-to-one relationship between the instances of Person entity type of unary relationship.

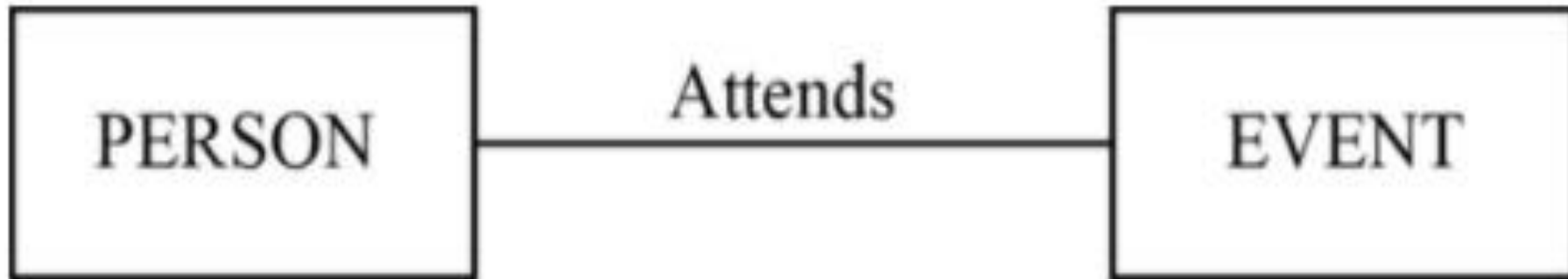


Binary relationship

- It is the relationship between the instances of two different entity types. Two entities will participate in the relationship.

Example

- Person and events are two different entity types which are related by using the relationship called “attends”.

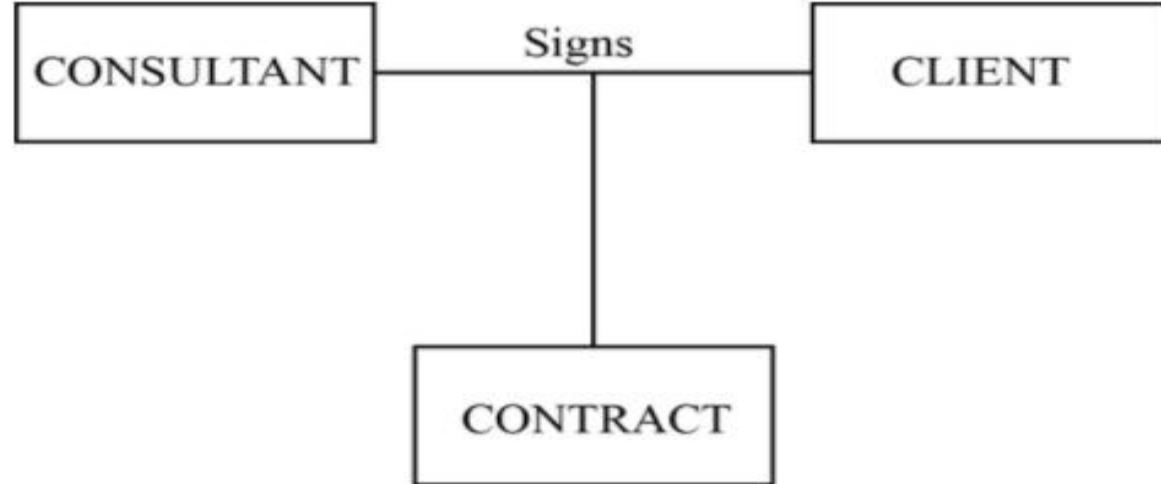


Ternary relationship

- A simultaneous relationship between the instances of three entity types with unique attributes is called a ternary relationship.

Example

- Consultant, client and contract are three different entities with different attributes.
- These three entities are related with a single relationship called “signs”.



ER EXAMPLE PROBLEM

Suppose you are given the following requirements for a simple database for the National Hockey League (NHL):

- The NHL has many teams,
- Each team has a ID, name, a city, a coach, a captain, and a set of players,
- Each player belongs to only one team,
- Each player has a ID, name, a position (such as left wing or goalie) and a skill level.
- A team captain is also a player,
- A game is played between two teams (referred to as host_team and guest_team)

and has a date (such as May 11th, 1999) and a score (such as 4 to 2).

Construct a clean and concise ER diagram for the NHL database.

IMPORTANT NOTATIONS FOR ERD



Represents Entity



Represents Attribute



Represents Relationship



Links Attribute(s) to entity set(s) or
Entity set(s) to Relationship set(s)



Represents Multivalued Attributes



Represents Derived Attributes

IMPORTANT NOTATIONS FOR ERD



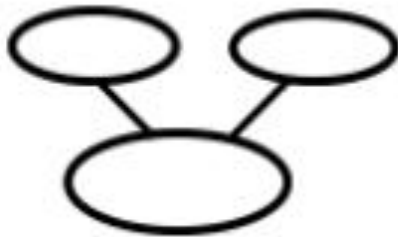
Represents Total Participation of Entity



Represents Weak Entity



Represents Weak Relationships



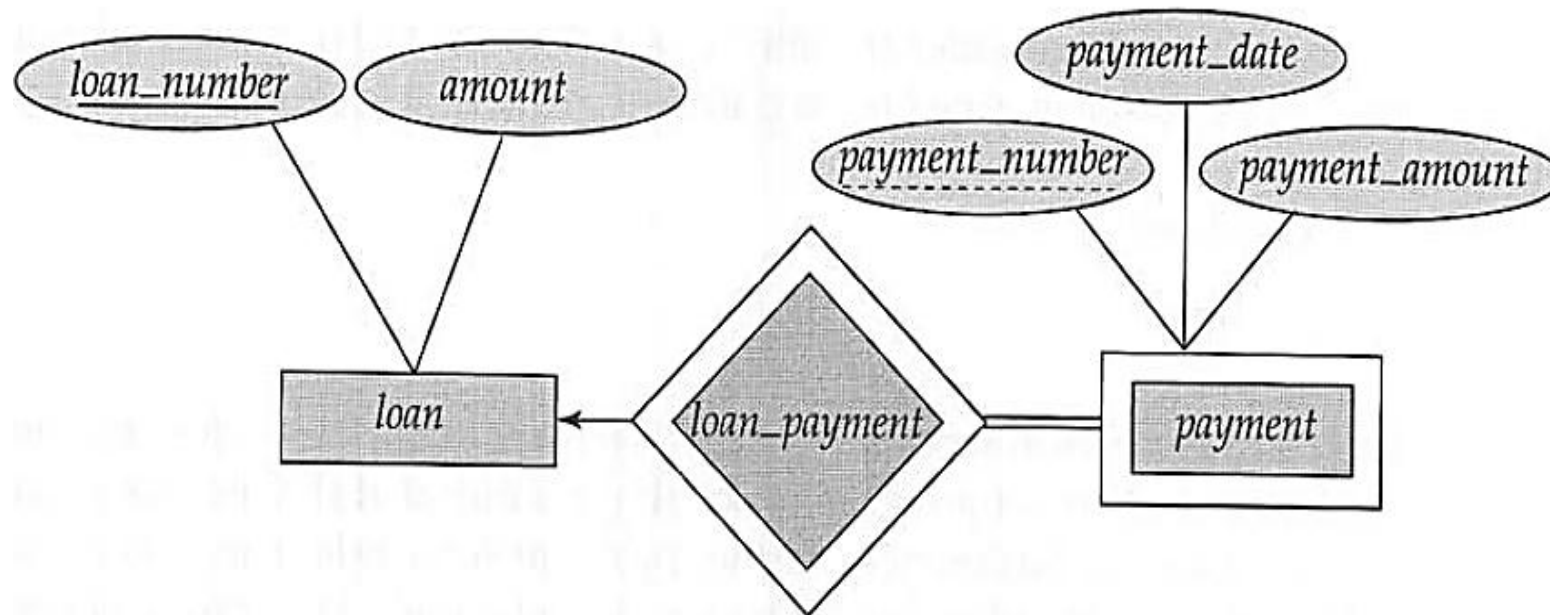
Represents Composite Attributes



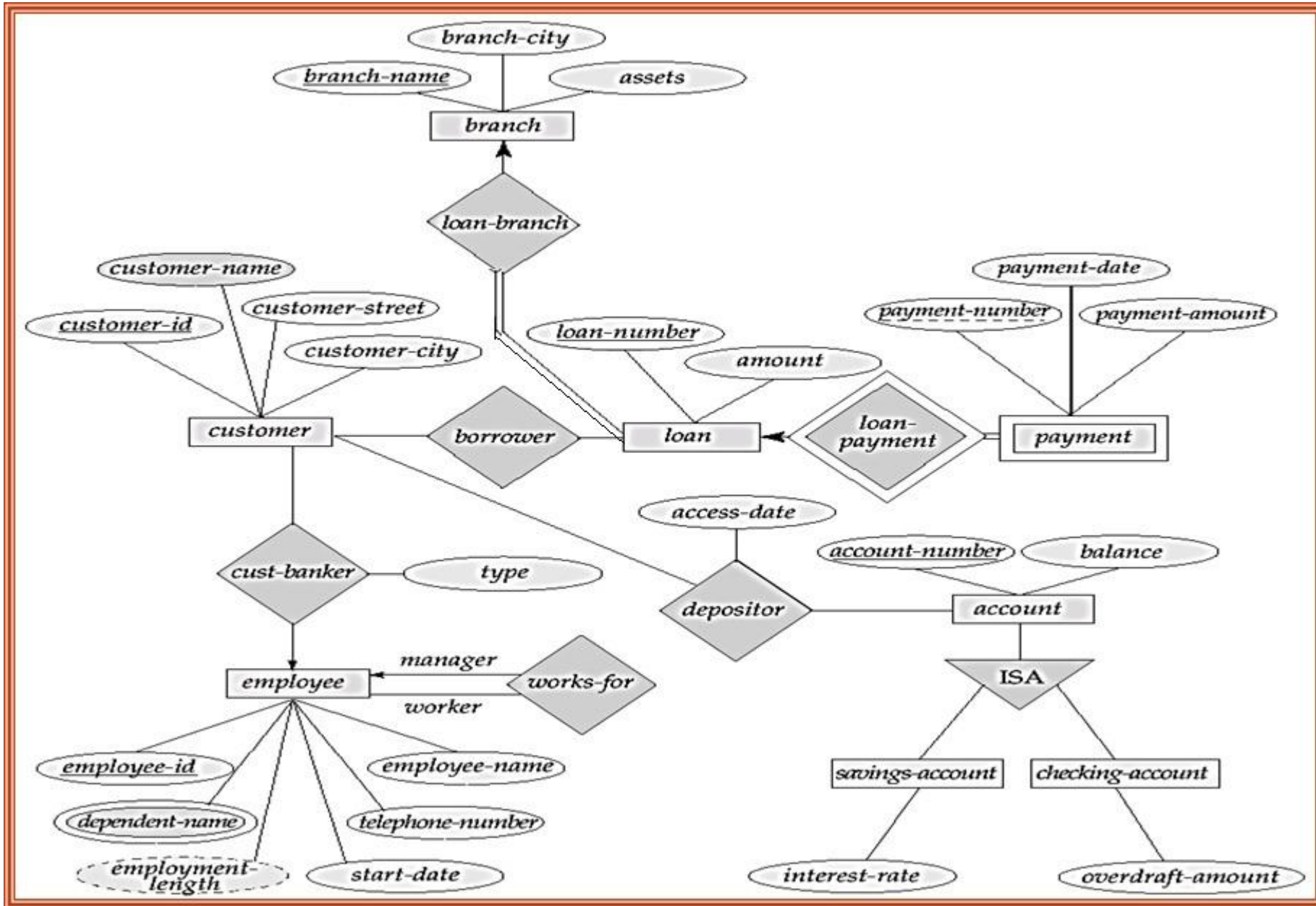
Represents Key Attributes / Single Valued Attributes

Weak Entity Sets

- The participation of the (weak) entity set *payment* in the relationship *loan* payments **total**,
 - meaning that every *payment* must be related via *loan_payment* to some *loan*.
- Finally, the arrow from *loan_payment* to *loan* indicates that each payment is for a single loan.
- The discriminator of a weak entity set also is underlined, but with a dashed, rather than a solid, line.



E-R diagram for a banking enterprise



SQL COMPONENTS

Statement	Type	Description
1.SELECT	Data retrieval	Retrieves data from the database
2.INSERT UPDATE DELETE	Data Manipulation (DML)	Enter new rows changes existing rows and removes unwanted rows from the tables in the database

SQL COMPONENTS

3.CREATE ALTER DROP RENAME TRUNCATE	Data Definition Language (DDL)	Setup changes and removes data structure from tables.
4.COMMIT ROLL BACK SAVEPOINT	Transaction control	Manages the changes made by DML statements.

SQL COMPONENTS

5.GRANT REVOKE	Data Control Language (DCL)	Gives and removes access rights to both oracle database and the structure with in it.
-------------------	-----------------------------------	--

DATA DEFINITION : Database Objects

- **Table:** Stores data
- **View:** Subset of data from one or more tables
- **Sequence:** Generates primary key values
- **Index:** Improves the performance of some queries
- **Synonym:** Gives alternative names to objects

CREATING A TABLE – ORDER PYTHON

```
CREATE TABLE [ORDER_PYTHON]
(
    [OrderId] TEXT PRIMARY KEY,
    [CustomerID] TEXT ,
    [OrderDate] TEXT,
    FOREIGN KEY([CustomerID]) REFERENCES [Customers3]([CustomerID])
)
```

CREATE TABLE QUERY

CREATE TABLE	Use to create a new table in Database
TEXT	Explaining the Datatype
PRIMARY KEY	Unique values column in the table – No Duplicate and Empty values
FOREIGN KEY	Use to Refer the column with some other table

ORDER PYTHON

<u>COLUMNS</u>	<u>KEYS</u>	<u>DATA TYPE</u>
ORDER ID CUSTOMER ID ORDER DATE	<u>PRIMARY KEY</u> ORDER ID <u>FOREIGN KEY</u> CUSTOMER ID REFEREING to ANOTHER COLUMN TABLE OF CUSTOMER3	TEXT for All Columns

CREATING ANOTHER TABLE – CUSTOMER PYTHON

```
CREATE TABLE [Customers_Python]  
([CustomerID] TEXT PRIMARY KEY ,  
    [CompanyName] TEXT,  
    [Address] TEXT,  
    [Phone] TEXT,  
    [Creditlimit] INTEGER CHECK (Creditlimit BETWEEN 0 AND 100000))
```

CUSTOMER PYTHON

<u>COLUMNS</u>	<u>KEYS</u>	<u>DATA TYPE</u>
CUSTOMER ID COMPANY NAME ADDRESS PHONE CREDITLIMIT	<u>PRIMARY KEY</u> CUSTOMER ID	TEXT for All Columns except CREDITLIMIT Column which has INETEGER TYPE and has check between 0 to 100000

Adding another Table in Database

```
CREATE TABLE [ORDERLINEITEM_PYTHON]
( [OrderId] TEXT ,
  [ProductID] TEXT,
  [OrderDate] TEXT,
  [CustomerID] TEXT,
  PRIMARY KEY (OrderId,ProductID),
  FOREIGN KEY([OrderId]) REFERENCES ORDER3 ([OrderId]),
  FOREIGN KEY ([CustomerID]) REFERENCES Customers3 ([CustomerID]))
```

ORDERLINEITEM_PYTHON

<u>COLUMNS</u>	<u>KEYS</u>	<u>DATA TYPE</u>
OrderId ProductID OrderDate CustomerID	<u>PRIMARY KEY</u> OrderId ProductID <u>FOREIGN KEY</u> - FOREIGN KEY([OrderId]) REFERENCES ORDER3 ([OrderId]) - FOREIGN KEY ([CustomerID]) REFERENCES Customers3 ([CustomerID]))	TEXT for All Columns

Adding Two New Tables in DataBase

```
CREATE TABLE [Employee]
(
    [empno] TEXT Primary key,
    [EmployeeName] TEXT,
    [Departno] TEXT,
    FOREIGN KEY ([Departno]) REFERENCES [department] ([Departno]) )
```

```
CREATE TABLE [department]
(
    [Departno] TEXT PRIMARY KEY,
    [DepartName] TEXT)
```

EMPLOYEE		
<u>COLUMNS</u>	<u>KEYS</u>	<u>DATA TYPE</u>
empno EmployeeName Departno	<u>PRIMARY KEY</u> empno <u>FOREIGN KEY</u> Departno REFEREING to ANOTHER TABLE COLUMN of department	TEXT for All Columns
DEPARTMENT		
<u>COLUMNS</u>	<u>KEYS</u>	<u>DATA TYPE</u>
Departno DepartName	<u>PRIMARY KEY</u> Departno	TEXT for All Columns

SQL DML operations

The SQL DML includes statement to perform following operations

STATEMENT	DESCRIPTION
INSERT	Enter new rows into tables
UPDATE	To change existing rows
DELETE	To delete existing rows

Inserting Values In Customers_Python Table

```
INSERT INTO Customers_Python VALUES (1,"John","ABC-XYZ","02134699",10000);
```

```
INSERT INTO Customers_Python VALUES (2,"Smith","ABC-XYZ","021344699",1000);
```

```
INSERT INTO Customers_Python VALUES (3,"Albert","ABC-XYZ","0213469",100);
```

CUSTOMER PYTHON

COLUMNS

CUSTOMER ID | COMPANY NAME | ADDRESS | PHONE | CREDITLIMIT

VALUES WILL BE INSERTED IN THREE ROWS

(1,"John","ABC-XYZ","02134699",10000);

(2,"Smith","ABC-XYZ","021344699",1000);

(3,"Albert","ABC-XYZ","0213469",100);

Note: Data save in the form of tuple in table , each value will be added in the respective columns

Inserting Values in Department Table

```
insert into department values(1, 'cis');
```

```
insert into department values(2, 'ccee');
```

```
insert into department values(3, 'textile');
```

Department

<u>COLUMNS</u>

Departno DepartName

<u>VALUES WILL BE INSERTED IN TWO ROWS</u>

(1, 'cis');

(2, 'ccee');

(3, 'textile');

```
insert into Employee VALUES (123,'John',1);  
insert into Employee VALUES (785,'Cart',1);  
insert into Employee VALUES (412,'Allen',2);  
insert into Employee VALUES (639,'Smith',3);
```

Employee

COLUMNS

empno | EmployeeName | Departno

VALUES WILL BE INSERTED IN TWO ROWS

```
(123,'John',1);  
(785,'Cart',1);  
(412,'Allen',2);  
(639,'Smith',3);
```

Updating an existing value from a Employee Table

Employee	
<u>QUERY</u>	<u>EXPLANATION</u>
update Employee set departno=2 where EmployeeName= 'Cart';	1. This tells the database that you want to modify data in the Employee table. 2. This specifies what column you want to change and its new value. Here, the Departno of the employee will be updated to 2 (which corresponds to the ccee department). 3. This is the condition. Only the rows that match this condition will be updated. Without a WHERE clause, all rows in the table would be updated!

Deleting an existing value from a Employee Table

Employee	
<u>QUERY</u>	<u>EXPLANATION</u>
DELETE FROM Employee WHERE empno=412;	1. This tells the database you want to remove rows from the Employee table. 2. This is the condition. Only the row(s) where empno equals 412 will be deleted. Without a WHERE clause, all rows in the Employee table would be deleted!

BASIC DATA RETRIEVAL

The Basic structure of an SQL query consists of three clause

SELECT

FROM

WHERE

SELECT *|{[DISTINCT] column | expression

FROM table

[**WHERE** condition];

Data Retrieval

<u>QUERY</u>	<u>EXPLANATION</u>
SELECT * FROM Employee, department;	1. The * means “select all columns” from the tables involved 2. This syntax lists two tables separated by a comma. In SQL, this creates a Cartesian product (also called a cross join) of the two tables. Cartesian product means every row in Employee is combined with every row in department, regardless of whether they match.

Result of the: SELECT * ,FROM Employee, department;

empno	EmployeeName	Departno	Departno	DepartName
123	John	1	1	cis
123	John	1	2	ccee
123	John	1	3	textile
785	Cart	2	1	cis
785	Cart	2	2	ccee
785	Cart	2	3	textile
639	Smith	3	1	cis
639	Smith	3	2	ccee
639	Smith	3	3	textile

Data Retrieval

<u>QUERY</u>	<u>EXPLANATION</u>
<pre>select * from Employee , department Where employee.Departno = department.Departno;</pre>	1. The * means “select all columns” from the tables involved 2. Lists two tables. Normally, this would create a Cartesian product (every row from Employee combined with every row from Department). 3. Adds a condition to only keep rows where the Departno in Employee matches the Departno in Department. This effectively performs an INNER JOIN between the two tables.

Data Retrieval Using JOIN

<u>QUERY</u>	<u>EXPLANATION</u>
<pre>select * from department LEFT OUTER JOIN Employee on Employee.Departno=departmen t.Departno;</pre>	1. The * means “select all columns” from the tables involved 2. A LEFT OUTER JOIN keeps all rows from the left table (department) even if there is no matching row in the right table (Employee). Rows from the right table (Employee) are included only if the Departno matches 3. This is the condition for matching rows. If an employee’s Departno matches a department’s Departno, the employee info is included. If a department has no employees, the employee columns will show NULL.

Data Retrieval Using MAX

<u>QUERY</u>	<u>EXPLANATION</u>
SELECT MAX(Departno)	1. MAX() is an aggregate function that returns the largest value in a column. Here, it finds the highest Departno in the department table.
FROM department;	2. Specifies the table from which to get the data.

Data Retrieval Using SUM – EXAMPLE 1

<u>QUERY</u>	<u>EXPLANATION</u>
SELECT SUM(Departno)	1. SUM() is an aggregate function that adds up all the values in a column. Here, it calculates the total of all Departno values in the department table
FROM department;	2. Specifies the table from which to get the data.

Data Retrieval Using SUM – EXAMPLE 2

<u>QUERY</u>	<u>EXPLANATION</u>
SELECT SUM(OrderID)	1. It returns a single total value — the sum of all order IDs. SUM() only works on numeric columns. If OrderId is defined as TEXT, the database cannot add them up unless you convert them to numbers.
FROM Order_Python;	2. Specifies the table from which to get the data.

Introduction To SQLite3

SQLite is a highly efficient, serverless, and self-contained SQL database engine that stands out for its simplicity and ease of integration. Designed to be embedded within applications, SQLite eliminates the need for separate database server processes and complex configurations.

- Imports the `sqlite3` module, which allows Python to work with SQLite databases.
- SQLite is a lightweight database that stores data in a simple `.db` file — no server needed.

```
[16... import sqlite3

conn = sqlite3.connect('example1.db')  # or your DB file name
cursor = conn.cursor()
```

=====:

conn = sqlite3.connect('example1.db')

- Connects your Python program to the database file named example.db.
- If this file doesn't exist, SQLite automatically creates it.
- The variable conn (short for connection) represents this connection to the database.

cursor = conn.cursor()

- Creates a cursor object, which is used to execute SQL commands (like SELECT, INSERT, UPDATE, etc.) inside the database.
- Think of it as a "pen" that can write or read inside the database.

=====:

```
# List all tables
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
print(cursor.fetchall())
```

```
# This means your database currently has one table:
# Customers
```

```
[('Customers',), ('sqlite_sequence',)]
```

```
=====
```

```
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
```

- Runs an SQL query that asks SQLite to list all table names in the database.
- `sqlite_master` is a special system table in SQLite that stores metadata about your database — including all tables, views, and indexes.
- The condition `WHERE type='table'` filters it to show only tables, not other objects.

```
print(cursor.fetchall())
```

- Retrieves all results from the query and prints them as a list of tuples.
- Each tuple contains one table name.

```
=====
```

```
[18... cursor.execute('''  
CREATE TABLE IF NOT EXISTS Customers (  
    CustomerID INTEGER PRIMARY KEY AUTOINCREMENT,  
    Name TEXT NOT NULL,  
    Email TEXT,  
    City TEXT  
)  
''')  
conn.commit()  
print("✅ Table 'Customers' created (if it didn't exist).")
```

✅ Table 'Customers' created (if it didn't exist).

=====

cursor.execute(""" ... ""')

- Sends the SQL command inside the triple quotes (""') to the database.
- The triple quotes make it easy to write multi-line SQL statements.

CREATE TABLE IF NOT EXISTS Customers (...)

- Creates a table named Customers only if it does not already exist — so it won't cause an error if the table is already there.

CustomerID INTEGER PRIMARY KEY AUTOINCREMENT

- CustomerID is a unique identifier for each row.
 - INTEGER PRIMARY KEY makes it the primary key.
 - AUTOINCREMENT automatically gives the next number to each new customer.
- =====

```
[20... cursor.execute("INSERT INTO Customers (Name, Email, City) VALUES (?, ?, ?)",  
                  ("Ali Khan", "ali@example.com", "Karachi"))  
cursor.execute("INSERT INTO Customers (Name, Email, City) VALUES (?, ?, ?)",  
                  ("Sara Ahmed", "sara@example.com", "Lahore"))  
conn.commit()
```

=====

cursor.execute("INSERT INTO Customers (Name, Email, City) VALUES (?, ?, ?)", (...))

- This command adds a new row into the Customers table.
- Name, Email, City are the columns you are providing values for.
- ?, ?, ? are placeholders for the actual values. This prevents SQL injection.

=====

```
[22... cursor.execute('SELECT * FROM Customers where Name="Ali Khan"')
rows = cursor.fetchall()
print(rows)
```

```
[(1, 'Ali Khan', 'ali@example.com', 'Karachi')]
```

=====

```
cursor.execute('SELECT * FROM Customers where Name="Ali Khan"')
```

- Executes a SQL query to get all rows from the Customers table where the Name is exactly "Ali Khan".
 - Only rows that match this condition will be returned
- =====

```
[23... # Close the connection  
conn.close()
```

=====

conn.close()

- Use to close the connection with the db file
- =====